Using LLMs via API

Owen Root

February 18, 2026

Web Interface vs API

- Two general platforms for interacting with LLMs:

Web Interface vs API

- Two general platforms for interacting with LLMs:
- Web Interface:

Web Interface vs API

- Two general platforms for interacting with LLMs:
- Web Interface:
  - Interact with LLM in a chat-bot like manner, within a website GUI.

Web Interface vs API

- Two general platforms for interacting with LLMs:
- Web Interface:
  - Interact with LLM in a chat-bot like manner, within a website GUI.
  - Access is based on subscriptions tiers, (i.e. $20 a month for ChatGPT Plus).

Web Interface vs API

- Two general platforms for interacting with LLMs:
- Web Interface:
  - Interact with LLM in a chat-bot like manner, within a website GUI.
  - Access is based on subscriptions tiers, (i.e. $20 a month for ChatGPT Plus).
  - Chats typically saved for later reference.

Web Interface vs API

- Two general platforms for interacting with LLMs:
- Web Interface:
  - Interact with LLM in a chat-bot like manner, within a website GUI.
  - Access is based on subscriptions tiers, (i.e. $20 a month for ChatGPT Plus).
  - Chats typically saved for later reference.
  - Limited control over LLM model and settings (varies between provider), options often tied to subscription tier.

Web Interface vs API

- Two general platforms for interacting with LLMs:
- Web Interface:
  - Interact with LLM in a chat-bot like manner, within a website GUI.
  - Access is based on subscriptions tiers, (i.e. $20 a month for ChatGPT Plus).
  - Chats typically saved for later reference.
  - Limited control over LLM model and settings (varies between provider), options often tied to subscription tier.
- API:

Web Interface vs API

- Two general platforms for interacting with LLMs:
- Web Interface:
  - Interact with LLM in a chat-bot like manner, within a website GUI.
  - Access is based on subscriptions tiers, (i.e. $20 a month for ChatGPT Plus).
  - Chats typically saved for later reference.
  - Limited control over LLM model and settings (varies between provider), options often tied to subscription tier.
- API:
  - API stands for Application Programming Interface. Allows for direct interaction with LLM via code (many programming languages supported).

Web Interface vs API

- Two general platforms for interacting with LLMs:
- Web Interface:
  - Interact with LLM in a chat-bot like manner, within a website GUI.
  - Access is based on subscriptions tiers, (i.e. $20 a month for ChatGPT Plus).
  - Chats typically saved for later reference.
  - Limited control over LLM model and settings (varies between provider), options often tied to subscription tier.
- API:
  - API stands for Application Programming Interface. Allows for direct interaction with LLM via code (many programming languages supported).
  - Greater access to LLMs and settings.

Web Interface vs API

- Two general platforms for interacting with LLMs:
- Web Interface:
  - Interact with LLM in a chat-bot like manner, within a website GUI.
  - Access is based on subscriptions tiers, (i.e. $20 a month for ChatGPT Plus).
  - Chats typically saved for later reference.
  - Limited control over LLM model and settings (varies between provider), options often tied to subscription tier.
- API:
  - API stands for Application Programming Interface. Allows for direct interaction with LLM via code (many programming languages supported).
  - Greater access to LLMs and settings.
  - Price is typically based on token usage (i.e. $1 per million tokens), so you only pay for what you use.

Web Interface vs API

- Two general platforms for interacting with LLMs:
- Web Interface:
  - Interact with LLM in a chat-bot like manner, within a website GUI.
  - Access is based on subscriptions tiers, (i.e. $20 a month for ChatGPT Plus).
  - Chats typically saved for later reference.
  - Limited control over LLM model and settings (varies between provider), options often tied to subscription tier.
- API:
  - API stands for Application Programming Interface. Allows for direct interaction with LLM via code (many programming languages supported).
  - Greater access to LLMs and settings.
  - Price is typically based on token usage (i.e. $1 per million tokens), so you only pay for what you use.
  - Better for automating LLM use.

Web Interface vs API

- Two general platforms for interacting with LLMs:
- Web Interface:
  - Interact with LLM in a chat-bot like manner, within a website GUI.
  - Access is based on subscriptions tiers, (i.e. $20 a month for ChatGPT Plus).
  - Chats typically saved for later reference.
  - Limited control over LLM model and settings (varies between provider), options often tied to subscription tier.
- API:
  - API stands for Application Programming Interface. Allows for direct interaction with LLM via code (many programming languages supported).
  - Greater access to LLMs and settings.
  - Price is typically based on token usage (i.e. $1 per million tokens), so you only pay for what you use.
  - Better for automating LLM use.
  - Downside of needing to write code to use.

Set Up for API usage

- Acquire API key from provider.

Set Up for API usage

- Acquire API key from provider.
  - API Key is a unique string that acts as a “password” to access LLMs from a given provider.

Set Up for API usage

- Acquire API key from provider.
  - API Key is a unique string that acts as a “password” to access LLMs from a given provider.
  - Usage and costs are tied to an API keys, so keep it secure.

Set Up for API usage

- Acquire API key from provider.
  - API Key is a unique string that acts as a “password” to access LLMs from a given provider.
  - Usage and costs are tied to an API keys, so keep it secure.
- Purchase usage credits for API key from provider.

Set Up for API usage

- Acquire API key from provider.
  - API Key is a unique string that acts as a "password" to access LLMs from a given provider.
  - Usage and costs are tied to an API keys, so keep it secure.
- Purchase usage credits for API key from provider.
  - for example, can buy $5 of credits. As LLM is used, this credit is depleted (generally based on the per token price).

Set Up for API usage

- Acquire API key from provider.
  - API Key is a unique string that acts as a "password" to access LLMs from a given provider.
  - Usage and costs are tied to an API keys, so keep it secure.
- Purchase usage credits for API key from provider.
  - for example, can buy $5 of credits. As LLM is used, this credit is depleted (generally based on the per token price).
  - CAREFULL: some LLM provider cut off access when credits are used, others will allowing you to go into "debt."

General API Usage Procedure

- General steps of procedure are similar, regardless of LLM or programming language used.

General API Usage Procedure

- General steps of procedure are similar, regardless of LLM or programming language used.
  1. Load LLM library, load API key.

General API Usage Procedure

- General steps of procedure are similar, regardless of LLM or programming language used.
  1. Load LLM library, load API key.
  2. Specify LLM and settings, send prompt to model.

General API Usage Procedure

- General steps of procedure are similar, regardless of LLM or programming language used.
  1. Load LLM library, load API key.
  2. Specify LLM and settings, send prompt to model.
  3. Process output and get results.

General API Usage Procedure

- General steps of procedure are similar, regardless of LLM or programming language used.
  1. Load LLM library, load API key.
  2. Specify LLM and settings, send prompt to model.
  3. Process output and get results.
- In Python, this looks like the following:

```python
import os
from openai import OpenAI

api_key = os.environ["OPENAI_API_KEY"] #loads API key, which is saved in a .env file
client = OpenAI(api_key=api_key)

response = client.responses.create( #specifies settings and sends prompt to LLM
    model="gpt-5.2",
    reasoning={"effort": "low"},
    input="Hello! How are you?",
)

print(response.output_text) # prints the LLMs response.
```

More Complex Usage

- Primary advantages of API usage are more direct control over model and model settings, and the ability to automation/additional processing around interacting with a model.

More Complex Usage

- Primary advantages of API usage are more direct control over model and model settings, and the ability to automation/additional processing around interacting with a model.
- Settings:

More Complex Usage

- Primary advantages of API usage are more direct control over model and model settings, and the ability to automation/additional processing around interacting with a model.
- Settings:
  - Different LLMs from different provider may have different setting options.

More Complex Usage

- Primary advantages of API usage are more direct control over model and model settings, and the ability to automation/additional processing around interacting with a model.
- Settings:
  - Different LLMs from different provider may have different setting options.
  - For example,

More Complex Usage

- Primary advantages of API usage are more direct control over model and model settings, and the ability to automation/additional processing around interacting with a model.
- Settings:
  - Different LLMs from different provider may have different setting options.
  - For example,
    - Some OpenAI models have a 'reasoning' parameter with options ("low", "medium", "high", "xhigh"), which controls "how long" the model spends thinking.

More Complex Usage

- Primary advantages of API usage are more direct control over model and model settings, and the ability to automation/additional processing around interacting with a model.
- Settings:
  - Different LLMs from different provider may have different setting options.
  - For example,
    - Some OpenAI models have a 'reasoning' parameter with options ("low", "medium", "high", "xhigh"), which controls "how long" the model spends thinking.
    - Whereas DeepSeek models instead have a 'thinking' parameter (True or False), which determines whether the models thinks or not.

More Complex Usage

- Primary advantages of API usage are more direct control over model and model settings, and the ability to automation/additional processing around interacting with a model.
- Settings:
  - Different LLMs from different provider may have different setting options.
  - For example,
    - Some OpenAI models have a 'reasoning' parameter with options ("low","medium","high","xhigh"), which controls "how long" the model spends thinking.
    - Whereas DeepSeek models instead have a 'thinking' parameter (True or False), which determines whether the models thinks or not.
  - Common "types" of settings are: control of thinking, temperature (controls "creativity"), system prompt (higher priority instructions), tool usage (such as using web search features).

More Complex Usage

- Primary advantages of API usage are more direct control over model and model settings, and the ability to automation/additional processing around interacting with a model.
- Settings:
  - Different LLMs from different provider may have different setting options.
  - For example,
    - Some OpenAI models have a 'reasoning' parameter with options ("low","medium","high","xhigh"), which controls "how long" the model spends thinking.
    - Whereas DeepSeek models instead have a 'thinking' parameter (True or False), which determines whether the models thinks or not.
  - Common "types" of settings are: control of thinking, temperature (controls "creativity"), system prompt (higher priority instructions), tool usage (such as using web search features).
- Additional processing example:

More Complex Usage

- Primary advantages of API usage are more direct control over model and model settings, and the ability to automation/additional processing around interacting with a model.
- Settings:
  - Different LLMs from different provider may have different setting options.
  - For example,
    - Some OpenAI models have a 'reasoning' parameter with options ("low","medium","high","xhigh"), which controls "how long" the model spends thinking.
    - Whereas DeepSeek models instead have a 'thinking' parameter (True or False), which determines whether the models thinks or not.
  - Common "types" of settings are: control of thinking, temperature (controls "creativity"), system prompt (higher priority instructions), tool usage (such as using web search features).
- Additional processing example:
  - LLM output typically includes the number ok tokens (and token types) used. Can use this, along with information about the per token cost for a model, to estimate the cost of a given LLM query.

Addition Processing Example - Query Costs

```
llmfs.openai_query(prompt='Hello! How are you today?')

#>>> I'm doing well—ready to help. What are you working on today?

#>>> Tokens — input: 13 (cached: 0), output: 20, total: 33
#>>> Estimated cost: $0.000303
#>>> That took 3.6345336437225340 seconds
```

Figure: Simple LLM query and response, showing estimate of costs from token counts

Using APIs Without Writing Code

- The standard use case for LLM APIs assumes programmatic access via code; however, there exist third-party applications provide graphical interfaces that accept an API key and handle all things “under the hood”.

Using APIs Without Writing Code

- The standard use case for LLM APIs assumes programmatic access via code; however, there exist third-party applications provide graphical interfaces that accept an API key and handle all things “under the hood”.
- These tools broadly fall into two categories:

Using APIs Without Writing Code

- The standard use case for LLM APIs assumes programmatic access via code; however, there exist third-party applications provide graphical interfaces that accept an API key and handle all things "under the hood".
- These tools broadly fall into two categories:
  - **Local (desktop) applications:** Run on your machine, data does not pass through a third-party server beyond the LLM provider itself.

Using APIs Without Writing Code

- The standard use case for LLM APIs assumes programmatic access via code; however, there exist third-party applications provide graphical interfaces that accept an API key and handle all things "under the hood".
- These tools broadly fall into two categories:
  - **Local (desktop) applications:** Run on your machine, data does not pass through a third-party server beyond the LLM provider itself.
    - **AnythingLLM** – feature-rich, supports multiple providers (OpenAI, Anthropic, Ollama, etc.), document ingestion, and local model support (only one I've tested).

Using APIs Without Writing Code

- The standard use case for LLM APIs assumes programmatic access via code; however, there exist third-party applications provide graphical interfaces that accept an API key and handle all things "under the hood".
- These tools broadly fall into two categories:
  - **Local (desktop) applications:** Run on your machine, data does not pass through a third-party server beyond the LLM provider itself.
    - **AnythingLLM** – feature-rich, supports multiple providers (OpenAI, Anthropic, Ollama, etc.), document ingestion, and local model support (only one I've tested).
    - **Jan** – lightweight, open-source, designed around local model inference (GGUF/llama.cpp) but also supports remote API providers.

Using APIs Without Writing Code

- The standard use case for LLM APIs assumes programmatic access via code; however, there exist third-party applications provide graphical interfaces that accept an API key and handle all things "under the hood".
- These tools broadly fall into two categories:
  - **Local (desktop) applications:** Run on your machine, data does not pass through a third-party server beyond the LLM provider itself.
    - **AnythingLLM** – feature-rich, supports multiple providers (OpenAI, Anthropic, Ollama, etc.), document ingestion, and local model support (only one I've tested).
    - **Jan** – lightweight, open-source, designed around local model inference (GGUF/llama.cpp) but also supports remote API providers.
    - **LM Studio** – primarily a local model runner, but includes an OpenAI-compatible local server and a clean chat UI.

Using APIs Without Writing Code

- The standard use case for LLM APIs assumes programmatic access via code; however, there exist third-party applications provide graphical interfaces that accept an API key and handle all things "under the hood".
- These tools broadly fall into two categories:
  - **Local (desktop) applications:** Run on your machine, data does not pass through a third-party server beyond the LLM provider itself.
    - **AnythingLLM** – feature-rich, supports multiple providers (OpenAI, Anthropic, Ollama, etc.), document ingestion, and local model support (only one I've tested).
    - **Jan** – lightweight, open-source, designed around local model inference (GGUF/llama.cpp) but also supports remote API providers.
    - **LM Studio** – primarily a local model runner, but includes an OpenAI-compatible local server and a clean chat UI.
  - **Browser / web-hosted applications:** Accessed through a browser; may self-host or use a managed service.

Using APIs Without Writing Code

- The standard use case for LLM APIs assumes programmatic access via code; however, there exist third-party applications provide graphical interfaces that accept an API key and handle all things "under the hood".
- These tools broadly fall into two categories:
  - **Local (desktop) applications:** Run on your machine, data does not pass through a third-party server beyond the LLM provider itself.
    - **AnythingLLM** – feature-rich, supports multiple providers (OpenAI, Anthropic, Ollama, etc.), document ingestion, and local model support (only one I've tested).
    - **Jan** – lightweight, open-source, designed around local model inference (GGUF/llama.cpp) but also supports remote API providers.
    - **LM Studio** – primarily a local model runner, but includes an OpenAI-compatible local server and a clean chat UI.
  - **Browser / web-hosted applications:** Accessed through a browser; may self-host or use a managed service.
    - **LibreChat** – open-source, self-hostable, supports multiple providers simultaneously, multi-agent and tool-use features.

Using APIs Without Writing Code

- The standard use case for LLM APIs assumes programmatic access via code; however, there exist third-party applications provide graphical interfaces that accept an API key and handle all things "under the hood".
- These tools broadly fall into two categories:
  - **Local (desktop) applications:** Run on your machine, data does not pass through a third-party server beyond the LLM provider itself.
    - **AnythingLLM** – feature-rich, supports multiple providers (OpenAI, Anthropic, Ollama, etc.), document ingestion, and local model support (only one I've tested).
    - **Jan** – lightweight, open-source, designed around local model inference (GGUF/llama.cpp) but also supports remote API providers.
    - **LM Studio** – primarily a local model runner, but includes an OpenAI-compatible local server and a clean chat UI.
  - **Browser / web-hosted applications:** Accessed through a browser; may self-host or use a managed service.
    - **LibreChat** – open-source, self-hostable, supports multiple providers simultaneously, multi-agent and tool-use features.
    - **TypingMind** – polished commercial interface, strong multi-model support, runs client-side (API key is not sent to TypingMind servers).

Using APIs Without Writing Code

- The standard use case for LLM APIs assumes programmatic access via code; however, there exist third-party applications provide graphical interfaces that accept an API key and handle all things "under the hood".
- These tools broadly fall into two categories:
  - **Local (desktop) applications:** Run on your machine, data does not pass through a third-party server beyond the LLM provider itself.
    - **AnythingLLM** – feature-rich, supports multiple providers (OpenAI, Anthropic, Ollama, etc.), document ingestion, and local model support (only one I've tested).
    - **Jan** – lightweight, open-source, designed around local model inference (GGUF/llama.cpp) but also supports remote API providers.
    - **LM Studio** – primarily a local model runner, but includes an OpenAI-compatible local server and a clean chat UI.
  - **Browser / web-hosted applications:** Accessed through a browser; may self-host or use a managed service.
    - **LibreChat** – open-source, self-hostable, supports multiple providers simultaneously, multi-agent and tool-use features.
    - **TypingMind** – polished commercial interface, strong multi-model support, runs client-side (API key is not sent to TypingMind servers).
    - **Open WebUI** – open-source, self-hostable, strong Ollama integration with broader API provider support added in recent versions.

Using APIs Without Writing Code – Considerations

- **Security is the primary concern.** Your API key authenticates and bills your account; any application that handles it warrants scrutiny.

Using APIs Without Writing Code – Considerations

- **Security is the primary concern.** Your API key authenticates and bills your account; any application that handles it warrants scrutiny.
  - Prefer applications that are open-source and auditable, or that have transparent, verifiable claims about how keys are handled.

Using APIs Without Writing Code – Considerations

- **Security is the primary concern.** Your API key authenticates and bills your account; any application that handles it warrants scrutiny.
  - Prefer applications that are open-source and auditable, or that have transparent, verifiable claims about how keys are handled.
  - Client-side key handling (key never leaves your browser or machine) is often safer than server-side handling by a third party.

Using APIs Without Writing Code – Considerations

- **Security is the primary concern.** Your API key authenticates and bills your account; any application that handles it warrants scrutiny.
  - Prefer applications that are open-source and auditable, or that have transparent, verifiable claims about how keys are handled.
  - Client-side key handling (key never leaves your browser or machine) is often safer than server-side handling by a third party.
  - Avoid entering API keys into applications with no established reputation or documentation.

Using APIs Without Writing Code – Considerations

- **Security is the primary concern.** Your API key authenticates and bills your account; any application that handles it warrants scrutiny.
  - Prefer applications that are open-source and auditable, or that have transparent, verifiable claims about how keys are handled.
  - Client-side key handling (key never leaves your browser or machine) is often safer than server-side handling by a third party.
  - Avoid entering API keys into applications with no established reputation or documentation.
- **Feature parity with the raw API is often incomplete.** Not all model parameters (e.g., reasoning effort, fine-grained sampling settings) are exposed in every UI; check whether the features you need are accessible.

Using APIs Without Writing Code – Considerations

- **Security is the primary concern.** Your API key authenticates and bills your account; any application that handles it warrants scrutiny.
  - Prefer applications that are open-source and auditable, or that have transparent, verifiable claims about how keys are handled.
  - Client-side key handling (key never leaves your browser or machine) is often safer than server-side handling by a third party.
  - Avoid entering API keys into applications with no established reputation or documentation.
- **Feature parity with the raw API is often incomplete.** Not all model parameters (e.g., reasoning effort, fine-grained sampling settings) are exposed in every UI; check whether the features you need are accessible.
- **Model and provider support varies.** Confirm the application supports the specific provider and model you intend to use before committing.

Using APIs Without Writing Code – Considerations

- **Security is the primary concern.** Your API key authenticates and bills your account; any application that handles it warrants scrutiny.
  - Prefer applications that are open-source and auditable, or that have transparent, verifiable claims about how keys are handled.
  - Client-side key handling (key never leaves your browser or machine) is often safer than server-side handling by a third party.
  - Avoid entering API keys into applications with no established reputation or documentation.
- **Feature parity with the raw API is often incomplete.** Not all model parameters (e.g., reasoning effort, fine-grained sampling settings) are exposed in every UI; check whether the features you need are accessible.
- **Model and provider support varies.** Confirm the application supports the specific provider and model you intend to use before committing.
- **Self-hosting is the most secure option for web-based tools** but requires some technical overhead (Docker, a server, etc.) – this may partially negate the appeal of avoiding code entirely.

Using APIs Without Writing Code – Considerations

- **Security is the primary concern.** Your API key authenticates and bills your account; any application that handles it warrants scrutiny.
  - Prefer applications that are open-source and auditable, or that have transparent, verifiable claims about how keys are handled.
  - Client-side key handling (key never leaves your browser or machine) is often safer than server-side handling by a third party.
  - Avoid entering API keys into applications with no established reputation or documentation.
- **Feature parity with the raw API is often incomplete.** Not all model parameters (e.g., reasoning effort, fine-grained sampling settings) are exposed in every UI; check whether the features you need are accessible.
- **Model and provider support varies.** Confirm the application supports the specific provider and model you intend to use before committing.
- **Self-hosting is the most secure option for web-based tools** but requires some technical overhead (Docker, a server, etc.) – this may partially negate the appeal of avoiding code entirely.
- In summary: viable options exist, but none are entirely without tradeoffs relative to direct API access.